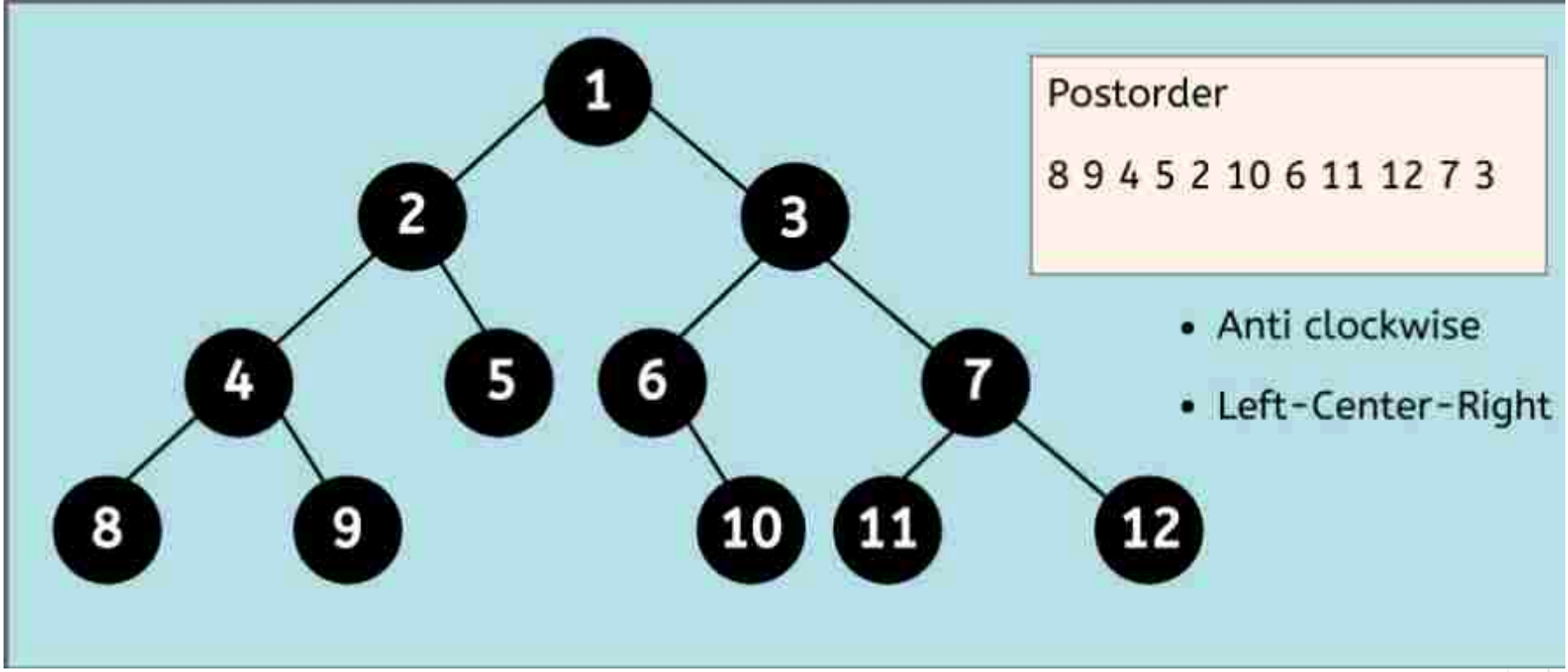
DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 64

Question description

Darsh, Ratik, Swathy are good friends, They are studying Pre-final year B.E. in reputed institution. Swathy's uncle was a DS teacher in school of computing technologies. He asks to make an application for identifying postorder Traversal of the Tree

You have to do the work for the development of the code of their thinking.

Functional Description



Constraints

T≤100
1≤ n ≤ 1000
0 ≤ *k* < *n*

Input Format
The first line of input consists of the number of test cases T. Then T lines follow each describing a test case. The first line of each test case consists of a number *n*. Then follow n lines each describing the friendship of a student. Each line starts with a number *k* telling the number of friends the *i*th person has. Then follow *k* integers separated by space which tells the index (1 based) of his friends. All these integers on the *i*th line are greater than *i* itself. (See sample for clarity).

Output Format
For each test case, if it is possible to divide the class into teams, then print a line of L's and R's denoting which team that student is a part of. If it is impossible to divide the class then print -1 instead. If there are multiple solutions print the one that would come first in the dictionary.

INPUT (STDIN)

```
7
90 70 50 75 130 110 150
```

EXPECTED OUTPUT

```
50 75 70 110 150 130 90
```

INPUT (STDIN)

```
9
50 23 56 89 43 75 130 110 150
```

EXPECTED OUTPUT

```
43 23 75 110 150 130 89 56 50
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
struct node
```

Test Case 2

KEYWORD

```
struct node *left,*right;
```

Test Case 3

KEYWORD

```
struct node* newNode(int item)
```

Test Case 4

KEYWORD

```
void postorder(struct node*
root)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
290
```

Test Case 3

NLOC

```
50
```

Code

solution.c

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

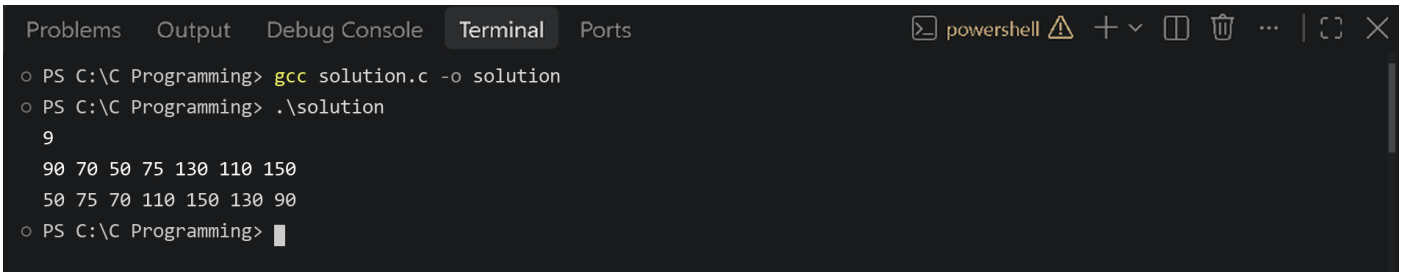
struct node* newNode(int item) {
    struct node* temp = (struct node*)malloc(sizeof(struct node));
    temp->data = item;
    temp->left = temp->right = NULL;
    return temp;
}

struct node* insert(struct node* node, int data) {
    if (node == NULL) return newNode(data);
    if (data < node->data) node->left = insert(node->left, data);
    else if (data > node->data) node->right = insert(node->right, data);
    return node;
}

void postorder(struct node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    int x;
    struct node* root = NULL;
    while (scanf("%d", &x) == 1) {
        root = insert(root, x);
    }
    postorder(root);
    printf("\n");
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
9
90 70 50 75 130 110 150
50 75 70 110 150 130 90
PS C:\C Programming>